

Подготовка к экзамену по C++ (ИТМО)

Тема 6: STL. Последовательные контейнеры

Hardcore Theory Edition

15 июня 2026 г.

1 Обзор последовательных контейнеров

В последовательных контейнерах порядок элементов зависит только от того, в каком порядке программист их туда добавил (в отличие от ассоциативных, где они сортируются сами).

1.1 `std::array` (Статический массив C++11)

Тонкая обертка над Си-массивом `int arr[10]`. Размер фиксируется **на этапе компиляции** (через `template`).

- **Плюсы:** Нулевой оверхед (`zero overhead`). Данные лежат строго на стеке. Нет никаких `new` и `delete`.
- **Минусы:** Размер нельзя изменить во время работы программы.

1.2 `std::vector` (Динамический массив)

Мы его уже детально разбирали. Память — один сплошной кусок в куче.

- **Золотое правило:** Используй `vector` всегда, пока профилировщик не докажет, что нужен другой контейнер.
- **Проблема:** Вставка в начало (`push_front`) отсутствует. Если сделать это через `insert(v.begin())`, весь вектор сдвинется вправо за $O(N)$.

1.3 `std::deque` (Двусторонняя очередь)

Читается как "дек" (Double-Ended Queue). Это гибрид вектора и списка.

- Внутри это **массив указателей на массивы** (`chunks`). Данные лежат не одним гигантским куском, а блоками (обычно по 4-8 КБ).
- **Суперсила:** Можно делать быстрое добавление как в конец (`push_back`), так и **в начало** (`push_front`) за $O(1)$. Компилятор просто выделяет новый блок памяти и добавляет указатель на него в управляющий массив.
- Поддерживает обращение по индексу `d[i]` за $O(1)$ (чуть медленнее вектора из-за двойного разыменования).

Суть на пальцах (Жизненная аналогия)

vector — это один длинный вагон метро. Если нужно добавить кресло в начало, придется сдвигать всех людей назад. **deque** — это поезд, состоящий из отдельных вагонов. Если нужно добавить место спереди, мы просто цепляем к поезду новый вагон спереди. Люди в старых вагонах остаются на своих местах!

1.4 `std::list` (Двусвязный список)

Каждый элемент выделяется в куче отдельным вызовом `new` и хранит два указателя (`prev` и `next`).

- **Минусы:** Огромный расход памяти (на 64-битной системе к каждому элементу добавляется 16 байт под указатели). Нельзя обратиться по индексу `l[5]`. Ужасная работа с кэшем процессора (Cache Misses).
- **Суперсила (`splice`):** Метод `splice` позволяет взять кусок одного списка и "перевесить" его в другой список за $O(1)$ **без копирования самих данных**. Мы просто перекидываем указатели узлов! У других контейнеров такого нет.

1.5 `std::forward_list` (Односвязный список)

То же самое, что и `list`, но указатель только один (`next`).

- Использует меньше памяти, но нельзя идти назад (`-it` не работает).
- Не имеет метода `.size()` (чтобы узнать размер, надо пройти весь список за $O(N)$).

2 Идиома Shrink-to-Fit

Когда вы удаляете элементы из вектора через `clear()` или `erase()`, размер (`size`) уменьшается, но зарезервированная память (`capacity`) **остается прежней** (чтобы не перевыделять память при новых добавлениях). Если вы хотите принудительно заставить вектор "похудеть" и отдать лишнюю память операционной системе, нужно вызвать метод `v.shrink_to_fit()`.